



المدرسة الوطنية للعلوم التطبيقية بتطوان
+ⵍⵎⵓⵔⵉⵏⵉ ⵜⴰⵎⵓⵔⵉⵜ ⵜⴰⵎⵓⵔⵉⵜ ⵜⴰⵎⵓⵔⵉⵜ
Ecole Nationale des Sciences Appliquées de Tétouan

Module : POO en JAVA

Filière : GI1

TP4 : Héritage, classes abstraites et Interfaces

Réalisée par : El Alaoui Hanane

Groupe de TP : 1

Code apogée : 23065496

Encadrée par : Prof EL BOUHDIDI Jaber

Exercice 1 :

La classe Figure:

```
public abstract class Figure { 3 inheritors
    protected int posX;
    protected int posY;
    public Figure(int posX, int posY) {
        this.posx = posX;
        this.posy = posY;}
    public int getPosX() { 1 override
        return posX;}
    public int getPosy() {
        return posY;}
    public void setPosX(int posX) { 1 override
        this.posx = posX;}
    public void setPosy(int posY) {
        this.posy = posY;}
    public double estDistantDe(Figure fig) {
        int dx = this.getPosX() - fig.getPosX();
        int dy = this.getPosy() - fig.getPosy();
        return Math.sqrt(dx*dx + dy*dy);}
    public void déplacement(int x, int y) {
        this.posx += x;
        this.posy += y;}
    public abstract double surface(); 2 implementations
    public abstract void afficher(); 3 implementations
}
```

La classe Rectangle :

```

public class Rectangle extends Figure implements Deformable{ 2 usages 1 inheritor
    protected double largeur; 4 usages
    protected double hauteur; 4 usages
    public Rectangle(int posX, int posY, double largeur, double hauteur) { 2 usages
        super(posx, posY);
        this.largeur = largeur; this.hauteur = hauteur;}
    public int getPosX() { 5 usages
        return posX;}
    public void setPosX(int posX) { no usages
        this.posx = posX;}
    public double getLargeur() { 1 usage
        return largeur;}
    public double getHauteur() { 1 usage
        return hauteur;}
    public void afficher(){ 1 usage 1 override
        System.out.println(posx + " " + posY + " " + getLargeur() + " " + getHauteur());}
    @Override no usages
    public double surface() {
        return largeur * hauteur;}
    @Override no usages
    public Figure deformation(double coeffH, double coeffV) {
        return new Rectangle(getPosX(), getPosY(), largeur: largeur * coeffH, hauteur: hauteur * coeffV);
    }
}

```

La classe Crre:

```

public class Carre extends Rectangle{ no usages
    public Carre(int posX,int posY ,double largeur, double hauteur) {
        super(posx,posy,largeur,hauteur);}
    public void afficher(){ 1 usage
        super.afficher();
    }
}

```

La classe Triangle:

```

public class Triangle extends Figure implements Deformable { 1 usage
    private double base; 6 usages
    private final double hauteur ; 4 usages
    public Triangle(int posX, int posY, double largeur, double hauteur) { 1 usage
        super(posX, posY);
        this.base = base;
        this.hauteur = hauteur;
    }
    public double getBase() { 1 usage
        return base;
    }
    public void setBase(double base) { no usages
        this.base = base;
    }
    @Override 1 usage
    public void afficher() {
        System.out.println(posX + " " + posY + " " + base + " " + hauteur);
    }
    @Override no usages
    public Figure deformation(double coeffH, double coeffV) {
        return new Triangle(getPosX(), getPosY(), largeur: getBase() * coeffH, hauteur: hauteur * coeffV);
    }
    @Override no usages
    public double surface() {
        return (base * hauteur) / 2;
    }
}

```

Interface Deformable:

```

public interface Deformable { 2 usages 3 implementations
    Figure deformation(double coeffH, double coeffV); no usages 2 implementations
}

```

Interface Triable :

```

public interface Triable { no usages 3 implementations

    // échange les éléments en positions i et j
    void échange(int i, int j); no usages 3 implementations

    // retourne vrai si l'élément en position i est plus grand que celui en position j
    boolean plusGrand(int i, int j); no usages 3 implementations

    // retourne le nombre d'éléments à trier
    int taille(); no usages 3 implementations
}

```

La classe TrieUtils :

```

public class TrieUtils {

    public static void triBulles(Triable t) { no usages
        boolean change;

        do {
            change = false;

            for (int i = 0; i < t.taille() - 1; i++) {
                if (t.plusGrand(i, j: i + 1)) {
                    t.echange(i, j: i + 1);
                    change = true;
                }
            }

        } while (change);
    }
}

```

La classe EntierTriable :

```
import java.util.Arrays;

public class EntierTriables implements Triable { no usages
    private int[] tab; 9 usages
    public EntierTriables(int[] tab) { no usages
        this.tab = tab;
    }
    @Override 1 usage
    public void exchange(int i, int j) {
        int tmp = tab[i];
        tab[i] = tab[j];
        tab[j] = tmp;
    }
    @Override 1 usage
    public boolean plusGrand(int i, int j) {
        return tab[i] > tab[j];
    }
    @Override 1 usage
    public int taille() {
        return tab.length;
    }
    @Override
    public String toString() {
        return Arrays.toString(tab);
    }
}
```

La classe DoublTriable:

```

import java.util.Arrays;

public class DoubleTriable implements Triable { 2 usages
    private double[] tab; 9 usages
    public DoubleTriable(double[] tab) { 1 usage
        this.tab = tab;
    }
    @Override 1 usage
    public void exchange(int i, int j) {
        double tmp = tab[i];
        tab[i] = tab[j];
        tab[j] = tmp;
    }
    @Override 1 usage
    public boolean plusGrand(int i, int j) {
        return tab[i] > tab[j];
    }
    @Override 1 usage
    public int taille() {
        return tab.length;
    }
    @Override
    public String toString() {
        return Arrays.toString(tab);
    }
}

```

La classe Dictionnaire :

```

import java.util.Arrays;
public class Dictionnaire implements Triable { 2 usages
    private String[] mots; 9 usages
    public Dictionnaire(String[] mots) { 1 usage
        this.mots = mots;
    }
    @Override 1 usage
    public void echange(int i, int j) {
        String tmp = mots[i];
        mots[i] = mots[j];
        mots[j] = tmp;
    }
    @Override 1 usage
    public boolean plusGrand(int i, int j) {
        return mots[i].compareTo(mots[j]) > 0; // ordre alphabétique
    }
    @Override 1 usage
    public int taille() {
        return mots.length;
    }
    @Override
    public String toString() {
        return Arrays.toString(mots);
    }
}

```

La classe testDoubTriable (Main):

```

public class TestDoubleTriable {
    public static void main(String[] args) {

        double[] t = {12.5, 3.2, 9.9, 1.1};

        DoubleTriable dt = new DoubleTriable(t);

        System.out.println("Avant tri : " + dt);

        TrieUtils.triBulles(dt);

        System.out.println("Après tri : " + dt);
    }
}

```

"C:\Program Files\Java\jdk-25\bin\java.exe"

Avant tri : [12.5, 3.2, 9.9, 1.1]

Après tri : [1.1, 3.2, 9.9, 12.5]

La classe TestDictionnaire(Main):

```

public class TestDictionnaire {
    public static void main(String[] args) {

        String[] tab = {"banane", "pomme", "abricot", "orange"};

        Dictionnaire d = new Dictionnaire(tab);

        System.out.println("Avant tri : " + d);

        TrieUtils.triBulles(d);

        System.out.println("Après tri : " + d);
    }
}

```

"C:\Program Files\Java\jdk-25\bin\java.exe" "-

Avant tri : [banane, pomme, abricot, orange]

Après tri : [abricot, banane, orange, pomme]

